MI ISP API

Version 3.4

REVISION HISTORY

Revision No.	Description	Date
3.0	• Initial release	12/04/2020

3.1	• Added new API:
	  • MI_ISP_SetCustSegAttr
	  • MI_ISP_GetCustSegAttr
	  • MI_ISP_GetCustSegBuf
	  • MI_ISP_PutCustSegBuf
	• Update API:
	  • MI_ISP_SetChnParam
		07/25/2021

	• Added PROCFS INTRODUCTION	08/25/2021
3.2	• Added new API: • MI_ISP_GetSubChnId	11/23/2021
3.3	• Added Mochi Info	02/17/2022
3.4	• Added Maruko info	03/21/2022

TABLE OF CONTENTS

1. OVERVIEW

1.1. Module description

ISP (Image Signal Processing) realizes HDR, 3D/2D noise reduction, 3A algorithm, WDR and other related functions.

1.2. Flow chart

1.2.1 Tiramisu

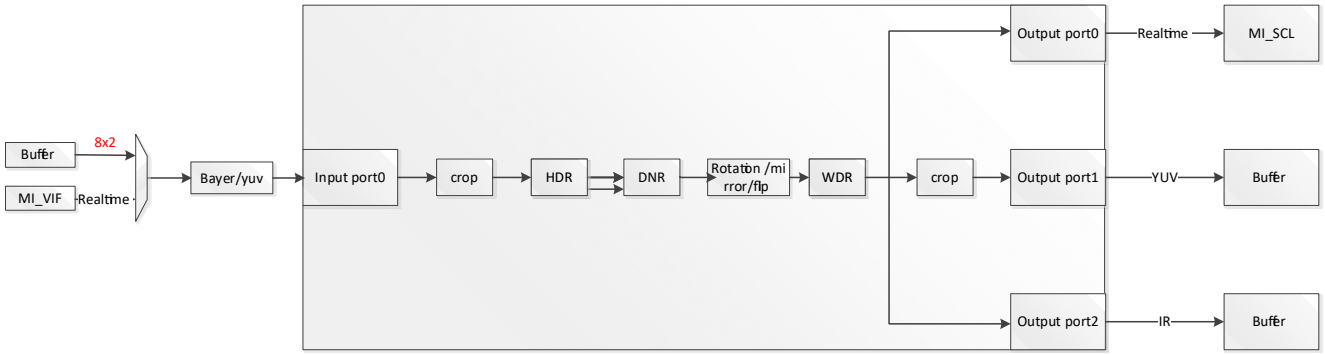


Figure 1-1: Tiramisu flow chart

※ Note

- Tiramisu only has a device0, and each device supports up to 16 channels.
- Only YUV422_yuyv / uyvy and Bayer format input are supported, Bayer format can support 3DNR and 3a functions.

1.2.2 Muffin

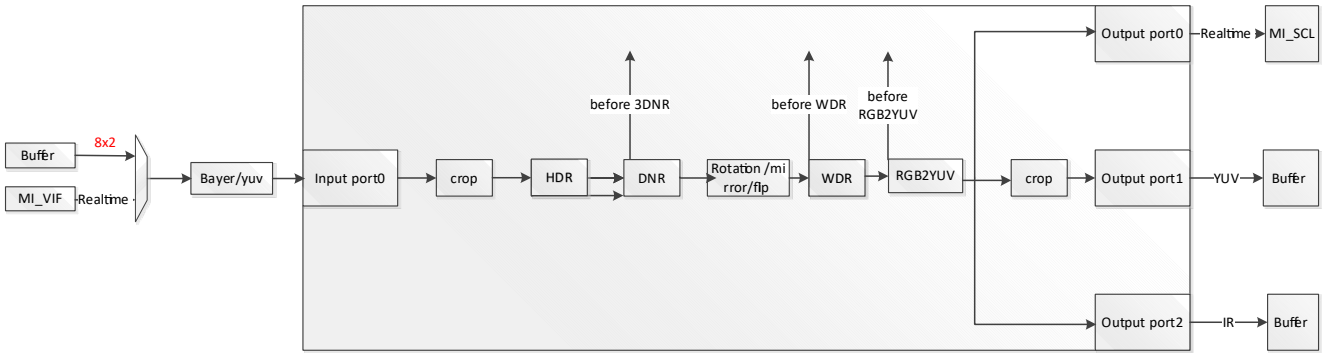


Figure 1-2 Muffin flow chart

※ Note

- Muffin has Device0 and Device1, and each Device supports up to 16 channels.
- Only supports YUV422_YUYV/UYVY and bayer format input.
- After enabling AI ISP through MI_ISP_SetCustSegAttr, it supports app to take out the buffer from

before 3DNR/before WDR/before RGB2YUV, and send it back to before 3DNR/before WDR/before RGB2YUV after processing.

- Before 3DNR is the data before entering the DNR module.
- AI ISP function means that app takes out the buffer from the MI ISP, uses an algorithm to optimize the image of the buffer, and then plugs it back into the MI ISP.

1.2.3 Mochi

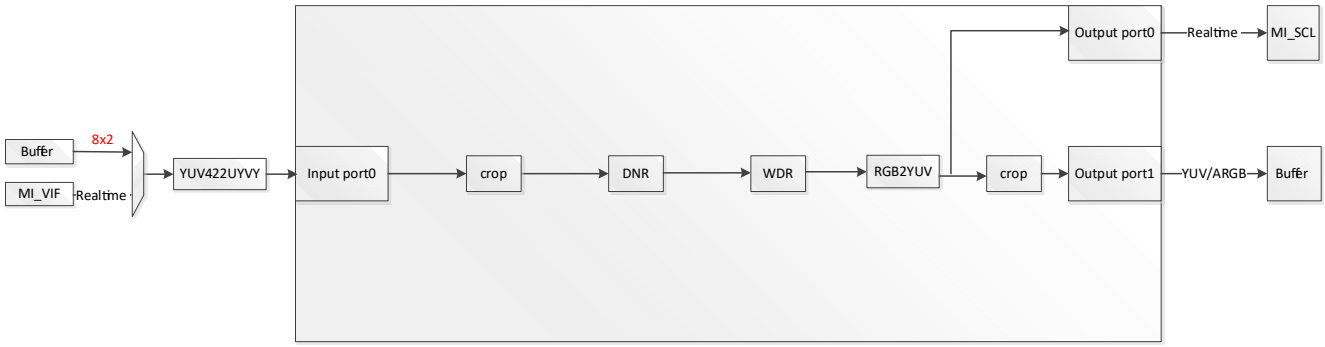


Figure 1-3: Mochi flow chart

※ Note

- Mochi only has a device0, and each device supports up to 16 channels.
- Only YUV422_uyvy format input are supported.
- Do not support HDR/Rotation/Mirror/Flip.

1.2.4 Maruko

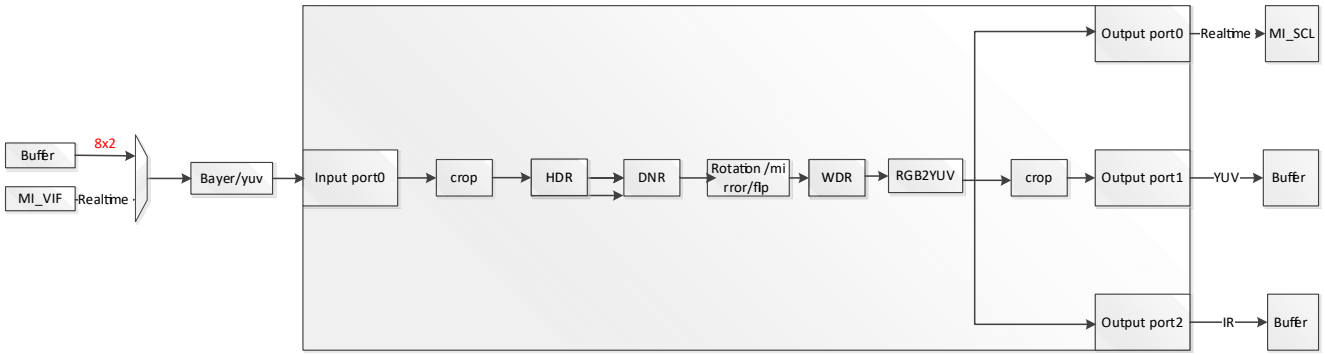


Figure 1-4: Maruko flow chart

※ Note

- Maruko only has a device0, and each device supports up to 16 channels.
- Only YUV422_yuyv / uyvy and Bayer format input are supported, Bayer format can support 3DNR/3A/Rot functions.

1.3. Keyword

- Device

Hardware device.

- channel
  Time division multiplexed channel on device.

- DNR
  Digital Noise Reduction.
  2D Noise Reduction: average a pixel with surrounding pixels, and the noise will be reduced after that, but the picture will be blurred;
  3D Noise Reduction: add time domain processing, 2D noise reduction only considers one frame of image, while 3D noise reduction considers the time domain relationship between frames, and averages each pixel in time domain.

- 3A algorithm
  AE (Auto Exposure), AWB (Auto White Balance), and AF(Auto Focus).

- HDR
  High-Dynamic Range.
  According to the LDR (Low-Dynamic Range) of different exposure times, the final HDR image is synthesized using the LDR image with the best detail corresponding to each exposure time. It can better reflect the visual effects in the real environment.

- WDR
  Wide Dynamic Range.
  After turning it on, the bright and dark parts of the scene can be seen clearly. The wide dynamic range is the ratio of the brightest and the darkest signal value that the image can distinguish.

- Rotation
  Rotate the original image around the center point by 0°/90°/180°/270°.

- mirror
  Horizontal mirror flip.

- Flip
  Mirror up and down.

- Crop
  Crop the input image.

- seg
  Segment module.

1.4. Insmod Parameter List

The parameters that can be carried after insmod mi_isp.ko are as follows:

Table 1-1 Parameter list

Parameter	Description	Value
g_u32DefaultDropNum	drop number of buffers after MI_ISP_CreateChannel	1.default:10 Because ISP needs to converge, ten buffers will drop by default. 2. Any value can be set by the user In the quick start scenario, reducing the number of drops can shorten the startup time.

2. API REFERENCE

This function module provides the following APIs:

API Name	Function
MI_ISP_CreateDevice	Create an ISP device
MI_ISP_DestoryDevice	Destroy an ISP device
MI_ISP_CreateChannel	Create an ISP channel
MI_ISP_DestroyChannel	Destroy an ISP channel
MI_ISP_SetInputPortCrop	Set ISP input port cropping parameter
MI_ISP_GetInputPortCrop	Get ISP input port cropping parameter
MI_ISP_SetChnParam	Set ISP channel parameter
MI_ISP_GetChnParam	Get ISP channel parameter
MI_ISP_StartChannel	Start ISP channel
MI_ISP_StopChannel	Stop ISP channel
MI_ISP_SetOutputPortParam	Set ISP output port parameter
MI_ISP_GetOutputPortParam	Get ISP output port parameter
MI_ISP_EnableOutputPort	Enable ISP output channel
MI_ISP_DisableOutputPort	Disable ISP output port
MI_ISP_Alloc_IQDataBuf	Allocate IQ data buffer
MI_ISP_Free_IQDataBuf	Release IQ data buffer
MI_ISP_CallBackTask_Register	Register ISP callback interface
MI_ISP_CallBackTask_Unregister	Unregister ISP callback interface
MI_ISP_SkipFrame	Set skip FrameNum
MI_ISP_LoadPortZoomTable	Load ISP port Zoom Table
MI_ISP_StartPortZoom	Start ISP port Zoom
MI_ISP_StopPortZoom	Stop ISP port Zoom
MI_ISP_GetPortCurZoomAttr	Get ISP port current Zoom attribute
MI_ISP_SetCustSegAttr	Set AI ISP attribute.
MI_ISP_GetCustSegAttr	Get AI ISP attribute
MI_ISP_GetCustSegBuf	Get AI ISP buffer
MI_ISP_PutCustSegBuf	Release AI ISP buffer

API Name	Function
MI_ISP_GetSubChnId	Get ISP subchannel ID

2.1. MI_ISP_CreateDevice

➢ Description

Create an ISP device.

➢ Syntax

MI_S32 MI_ISP_CreateDevice(MI_ISP_DEV DevId, MI_ISP_DevAttr_t *pstDevAttr);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID	Input
pstDevAttr	ISP device attribute pointer. Static attribute.	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

- Make sure that the ISP device is disabled before calling. If it is enabled, please use MI_ISP_DestoryDevice to disable it.

➢ Example

MI_ISP initialization:

```
MI_S32 ST_IspModuleInit(MI_ISP_DEV IspDevId)
{
#define ST_MAX_ISP_OUTPORT_NUM (2)
   MI_ISP_CHANNEL IspChnId = 0;
   MI_ISP_PORT  IspOutPortId =0;

      MI_ISP_DevAttr_t stCreateDevAttr;
      memset(&stCreateDevAttr, 0x0, sizeof(MI_ISP_DevAttr_t));
      stCreateDevAttr.u32DevStitchMask = E_MI_ISP_DEVICE_ID0;

      MI_ISP_CreateDevice(IspDevId, &stCreateDevAttr);

      MI_ISP_ChannelAttr_t  stIspChnAttr;
      memset(&stIspChnAttr, 0x0, sizeof(MI_ISP_ChannelAttr_t));
    stIspChnAttr. u32SensorBindId = E_MI_ISP_SENSOR0;
      MI_ISP_CreateChannel(IspDevId, IspChnId, &stIspChnAttr);
```

```
    MI_SYS_WindowRect_t  stInputCropInfo;
      stInputCropInfo.u16x=0;
      stInputCropInfo.u16y=0;
      stInputCropInfo.u16width=0;
      stInputCropInfo.u16height=0;//width/height==0 no use crop.
      MI_ISP_SetInputPortCrop(IspDevId, IspChnId, &stInputCropInfo);

    MI_ISP_ChnParam_t stChnParam;
    stChnParam.eHDRType = E_MI_ISP_HDR_TYPE_OFF;
    stChnParam.e3DNRLevel = E_MI_ISP_3DNR_LEVEL2;
    stChnParam.bMirror = FALSE;
    stChnParam.bFlip = FALSE;
    stChnParam.eRot = E_MI_SYS_ROTATE_NONE;
      //use rot or flip, 3dnr level must more than 0
    stChnParam.bY2bEnable = FALSE;
      MI_ISP_SetChnParam(IspDevId, IspChnId, &stChnParam);
      MI_ISP_StartChannel(IspDevId, IspChnId);

      for(IspOutPortId=0; IspOutPortId<ST_MAX_ISP_OUTPORT_NUM; IspOutPortId++)
      {
            MI_ISP_OutPortParam_t  stIspOutputParam;
            memset(&stIspOutputParam, 0x0, sizeof(MI_ISP_OutPortParam_t));
          stIspOutputParam.stCropRect.u16x=0;
        stIspOutputParam.stCropRect.u16y=0;
        stIspOutputParam.stCropRect.u16Width=0;
        stIspOutputParam.stCropRect.u16Height=0;//width/height use 0, not use crop
            stIspOutputParam.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_YUV420;
            MI_ISP_SetOutputPortParam(IspDevId, IspChnId, IspOutPortId,
&stIspOutputParam);
            MI_ISP_EnableOutputPort(IspDevId, IspChnId, IspOutPortId);
      }

   return MI_SUCCESS;
}
```

MI_ISP de-initialization:

```
MI_S32 ST_IspModuleUnInit(MI_ISP_DEV IspDevId)
{
#define ST_MAX_ISP_OUTPORT_NUM (2)
    MI_ISP_CHANNEL IspChnId = 0;
    MI_ISP_PORT  IspOutPortId =0;
```

```
    for(IspOutPortId=0; IspOutPortId<ST_MAX_ISP_OUTPORT_NUM; IspOutPortId++)
    {
        MI_ISP_DisableOutputPort(IspDevId, IspChnId, IspOutPortId);
    }

    MI_ISP_StopChannel(IspDevId, IspChnId);
    MI_ISP_DestroyChannel(IspDevId, IspChnId);

    MI_ISP_DestoryDevice(IspDevId);

    return MI_SUCCESS;
}
```

- Related APIs

  MI_ISP_DestoryDevice

2.2. MI_ISP_DestoryDevice

- Description

  Destroy an ISP device.

- Syntax

  MI_S32 MI_ISP_DestoryDevice(MI_ISP_DEV DevId);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note

You need to disable all the output ports first, and then stop and destroy the channel to destroy device.

- Example

  See the example of MI_ISP_CreateDevice.

- Related APIs

MI_ISP_CreateDevice

2.3. MI_ISP_CreateChannel

➢ Description

Create an ISP channel.

➢ Syntax

MI_S32 MI_ISP_CreateChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_ChannelAttr_t *pstChAttr);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstChAttr	ISP channel attribute pointer	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

Create device firstly, and then create channel.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

MI_ISP_DestroyChannel.

2.4. MI_ISP_DestroyChannel

➢ Description

Destroy an ISP channel.

➢ Syntax

MI_S32 MI_ISP_DestroyChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

You need to disable all the output ports first, and then stop channel, so that you can destroy channel.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

MI_ISP_CreateChannel

2.5. MI_ISP_SetInputPortCrop

➢ Description

Set ISP input port cropping parameter.

➢ Syntax

MI_S32 MI_ISP_SetInputPortCrop(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_SYS_WindowRect_t *pstCropInfo);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstCropInfo	ISP input port cropping parameter pointer	Input

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

- When the input is Dram buffer, the input crop can be supported.
- Crop x, Crop y, width, height are all aligned with 2.
- The 3A inside the Isp only counts the information of the cropped area, which will affect the final 3A calculation result of the entire screen.
- When the input data compression mode is not E_MI_SYS_COMPRESS_MODE_NONE, the input data does not support cropping.

➢ Example

None.

➢ Related APIs

None.

2.6. MI_ISP_GetInputPortCrop

➢ Description

Get ISP input port cropping parameter.

➢ Syntax

```
MI_S32 MI_ISP_GetInputPortCrop(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_SYS_WindowRect_t *pstCropInfo);
```

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstCropInfo	ISP input port cropping parameter pointer	output

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

None.

➢ Example

None.

➢ Related APIs

None.

2.7. MI_ISP_SetChnParam

➢ Description

Set ISP channel parameter.

➢ Syntax

MI_S32 MI_ISP_SetChnParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_ChnParam_t *pstChnParam);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstChnParam	ISP channel parameter pointer	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

It is recommended to switch the channel parameters when the channel is stopped to avoid mismatch between the buffer and back-end parameters in the channel, which may cause an exception.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

MI_ISP_GetChnParam

2.8. MI_ISP_GetChnParam

➢ Description

Get ISP channel parameter.

➢ Syntax

MI_S32 MI_ISP_GetChnParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_ChnParam_t

*pstChnParam);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstChnParam	ISP channel parameter pointer	output

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note

None.

- Example

None.

- Related APIs

MI_ISP_SetChnParam

2.9. MI_ISP_StartChannel

- Description

Start ISP channel.

- Syntax

MI_S32 MI_ISP_StartChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h

- Library: libmi_isp.a

※ Note

None.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

MI_ISP_SetChnParam

2.10. MI_ISP_StopChannel

➢ Description

Stop ISP channel.

➢ Syntax

MI_S32 MI_ISP_StopChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

You can stop the channel when created, and all the output port will have no data output.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

None.

2.11. MI_ISP_SetOutputPortParam

➢ Description

Set ISP output port parameter.

- Syntax

  MI_S32 MI_ISP_SetOutputPortParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId, MI_ISP_OutPortParam_t *pstOutPortParam);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
PortId	ISP port ID	Input
pstOutPortParam	ISP output port parameter pointer	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note
  - Output port0 can only bind with MI_SCL realtime, output area is the same as input, which has no need to call this API to set.
  - Output port1 support crop, and the output size need to be specified.
  - Output port2 only supports the output of the IR data of the RGBIR sensor. Output W/H is fixed at half of the input W/H, which has no need to call this API to set.

- Example

  See the example of MI_ISP_CreateDevice.

- Related APIs

  None.

2.12. MI_ISP_GetOutputPortParam

- Description

  Get ISP output port parameter.

- Syntax

  MI_S32 MI_ISP_GetOutputPortParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId, MI_ISP_OutPortParam_t *pstOutPortParam);

- Parameter

Parameter name	Description	Input/Output

DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
PortId	ISP port ID	Input
pstOutPortParam	ISP output port parameter pointer	output

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

None.

➢ Example

None.

➢ Related APIs

MI_ISP_SetOutputPortParam

2.13. MI_ISP_EnableOutputPort

➢ Description

Enable ISP output channel.

➢ Syntax

MI_S32 MI_ISP_EnableOutputPort(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
PortId	ISP port ID	Input

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h

- Library: libmi_isp.a

※ Note

- Output port1 can be enabled only when the attribute is set.
- Output port2 can only be enabled when using RGBIR sensor.

➢ Example

See the example of MI_ISP_CreateDevice.

➢ Related APIs

MI_ISP_DisableOutputPort

2.14. MI_ISP_DisableOutputPort

➢ Description

Disable ISP output port.

➢ Syntax

MI_S32 MI_ISP_DisableOutputPort(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
PortId	ISP port ID	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

None.

➢ Example

None.

➢ Related APIs

None.

2.15. MI_ISP_Alloc_IQDataBuf

- Description

  Allocate IQ data buffer.

- Syntax

  MI_S32 MI_ISP_Alloc_IQDataBuf(MI_U32 u32Size,void **pUserVirAddr);

- Parameter

Parameter name	Description	Input/Output
u32Size	IQ data buffer size.	Input
pUserVirAddr	IQ data buffer pointer address	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note
  - It is recommended to use this api to apply for IQ api data to avoid the high risk of cpu loading caused by high frequency calls.
  - A thread applies for an isp data buffer to avoid buffer sharing between threads.

- Example

```
#define  MI_ISP_MAX_DATA_SIZE  (80*1024)
MI_ISP_Alloc_IQDataBuf(MI_ISP_MAX_DATA_SIZE, &pIspBuffer);
MI_ISP_IQ_ColorToGrayType_t *pstColorToGray = (MI_ISP_IQ_ColorToGrayType_t
*)pIspBuffer;
MI_ISP_IQ_GetColorToGray( Channel, pstColorToGray);
if(pstColorToGray->bEnable == SS_TRUE)
      pstColorToGray->bEnable = SS_FALSE;
else
      pstColorToGray->bEnable = SS_TRUE;
MI_ISP_IQ_SetColorToGray( Channel, pstColorToGray);

MI_ISP_IQ_ContrastType_t *pstContrast = (MI_ISP_IQ_ContrastType_t *)pIspBuffer;
MI_ISP_IQ_GetContrast( Channel, pstContrast);
MI_ISP_IQ_SetContrast( Channel, pstContrast);

MI_ISP_Free_IQDataBuf(pIspBuffer)
```

➢ Related APIs

MI_ISP_Free_IQDataBuf.

2.16. MI_ISP_Free_IQDataBuf

➢ Description

Release IQ data buffer.

➢ Syntax

MI_S32 MI_ISP_Free_IQDataBuf(void *pUserBuf);

➢ Parameter

Parameter name	Description	Input/Output
pUserBuf	IQ request data buffer pointer	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

None.

➢ Example

See the example of MI_ISP_Alloc_IQDataBuf.

➢ Related APIs

None.

2.17. MI_ISP_CallBackTask_Register

➢ Description

Register ISP callback interface.

➢ Syntax

MI_S32 MI_ISP_CallBackTask_Register(MI_ISP_DEV DevId, MI_ISP_CallBackParam_t *pstCallBackParam);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input

pstCallBackParam	ISP callback parameter	Input

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note
- The interface currently only supports calling by kernel mode.
- Called in pair with MI_ISP_CallBackTask_Unregister.

➢ Example

```
MI_S32 _MI_ISP_vsync1(MI_U64 u64Data)
{
   DBG_ERR("DATA %llu \n", u64Data);
   return 0;
}

MI_S32 _MI_ISP_vsync2(MI_U64 u64Data)
{
   DBG_ERR("DATA %llu \n", u64Data);
   return 0;
}

static MS_S32 _MI_ISP_testRegCallback(void)
{
   MI_ISP_CallBackParam_t stCallBackParam1;
   MI_ISP_CallBackParam_t stCallBackParam2;
   memset(&stCallBackParam1, 0x0, sizeof(MI_ISP_CallBackParam_t));
   memset(&stCallBackParam2, 0x0, sizeof(MI_ISP_CallBackParam_t));
   stCallBackParam1.eCallBackMode = E_MI_ISP _CALLBACK_ISR;
   stCallBackParam1. eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
   stCallBackParam1.pfnCallBackFunc = _mi_ISP_vsync1;
   stCallBackParam1.u64Data = 12;
   MI_ISP_CallBackTask_Register(&stCallBackParam1);
   stCallBackParam2.eCallBackMode = E_MI_ISP_CALLBACK_ISR;
   stCallBackParam2. eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
   stCallBackParam2.pfnCallBackFunc = _mi_ISP_vsync2;
   stCallBackParam2.u64Data = 23;
   MI_ISP_CallBackTask_Register(&stCallBackParam2);
   return 0;
```

```
}

static MS_S32 _MI_ISP_testUnRegCallback(void)
{
    MI_ISP_CallBackParam_t stCallBackParam1;
    MI_ISP_CallBackParam_t stCallBackParam2;
    memset(&stCallBackParam1, 0x0, sizeof(MI_ISP_CallBackParam_t));
    memset(&stCallBackParam2, 0x0, sizeof(MI_ISP_CallBackParam_t));
    stCallBackParam1.eCallBackMode = E_MI_ISP_CALLBACK_ISR;
    stCallBackParam1.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallBackParam1.pfnCallBackFunc = _mi_ISP_vsync1;
    stCallBackParam1.u64Data = 12;
    MI_ISP_CallBackTask_Unregister(&stCallBackParam1);
    stCallBackParam2.eCallBackMode = E_MI_ISP_CALLBACK_ISR;
    stCallBackParam2.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallBackParam2.pfnCallBackFunc = _mi_ISP_vsync2;
    stCallBackParam2.u64Data = 23;
    MI_ISP_CallBackTask_Unregister(&stCallBackParam2);
    return 0;
}
```

- Related APIs

  MI_ISP_CallBackTask_Unregister

2.18. MI_ISP_CallBackTask_Unregister

- Description

  Unregister ISP callback interface

- Syntax

  MI_S32 MI_ISP_CallBackTask_Unregister(MI_ISP_DEV DevId, MI_ISP_CallBackParam_t *pstCallBackParam);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
pstCallBackParam	ISP callback parameter	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note
- The interface currently only supports calling by kernel mode.
- Called in pair of MI_ISP_CallBackTask_Register.

➢ Example
See the example of MI_ISP_CallBackTask_Register

➢ Related APIs
None.

2.19. MI_ISP_SkipFrame

➢ Description
Set to skip Frame count from calling this API.

➢ Syntax
MI_S32 MI_ISP_SkipFrame(MI_ISP_DEV DevId,MI_ISP_CHANNEL ChnId,
MI_U32 u32FrameNum);

➢ Parameter

Parameter name	Description	Input/Output
ChnId	ISP channel ID	Input
u32FrameNum	Frame number	Input

➢ Return value
- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence
- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note
None.

➢ Example
None.

➢ Related APIs
None.

2.20. MI_ISP_LoadPortZoomTable

➢ Description

Load ISP port Zoom Table.

➢ Syntax

MI_S32 MI_ISP_LoadPortZoomTable(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_ZoomTable_t *pZoomTable);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pZoomTable	Zoom Table parameter	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

- Load zoom table through the interface in advance, each frame applies the zoom parameters in the table in turn to ensure smooth zoom effects.
- Call MI_ISP_SetOutputPortParam cyclically to set crop, which cannot guarantee the effective setting of each frame and cannot achieve smooth effect.

X0	Y0	W0	H0	SensorId=0
X1	Y1	W1	H1	SensorId=0
X2	Y2	W2	H2	SensorId=1
X3	Y3	W3	H3	SensorId=1

The table contains crop position and corresponding sensorid information, sensorid information supports long and short focus switching scenes.

- The zoom function is completed by the cooperation of MI_ISP and MI_SCL. MI_ISP is responsible for the 3A effect and the long and short focus switching scenes during the zoom process to notify the sensor driver to switch the sensor. MI_SCL is responsible for cropping and scaling up, so in the zoom scene, MI_SCL can no longer use the crop function.
- During the operation of Zoom, you cannot load the table repeatedly, please use MI_ISP_StopPortZoom first, and then load table.

➢ Example

None.

➢ Related APIs

None.

2.21. MI_ISP_StartPortZoom

➢ Description

Start ISP port Zoom.

➢ Syntax

MI_S32 MI_ISP_StartPortZoom(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_ZoomAttr_t *pstZoomAttr);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstZoomAttr	Zoom Table parameter	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

X0	Y0	W0	H0	SensorId=0
X1	Y1	W1	H1	SensorId=0
X2	Y2	W2	H2	SensorId=1
X3	Y3	W3	H3	SensorId=1

u32FromEntryIndex and u32ToEntryIndex are any one of index 0~3 in the table.
When u32FromEntryIndex != u32ToEntryIndex, apply crop parameters from u32FromEntryIndex to u32ToEntryIndex frame by frame.
When u32FromEntryIndex == u32ToEntryIndex, it will stop at the current u32FromEntryIndex parameter.
When MI detects that the sensorid of the next entry is inconsistent with the current sensor id, the sensor driver will be notified of the sensor driver after the current frame is finished, and switch to the corresponding sensor.

➢ Example

None.

➢ Related APIs

MI_ISP_StopPortZoom

2.22. MI_ISP_StopPortZoom

➢ Description

Stop ISP port Zoom.

➢ Syntax

MI_S32 MI_ISP_StopPortZoom(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

Called in pair with MI_ISP_StartPortZoom.

➢ Example

None.

➢ Related APIs

None.

2.23. MI_ISP_GetPortCurZoomAttr

➢ Description

Get ISP port current Zoom attribute.

➢ Syntax

MI_S32 MI_ISP_GetPortCurZoomAttr(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_ISP_ZoomAttr_t *pstZoomAttr);

➢ Parameter

Parameter name	Description	Input/Output

DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstZoomAttr	Zoom attribute	output

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note

None.

- Example

None.

- Related APIs

None.

2.24. MI_ISP_SetCustSegAttr

- Description

Set AI ISP attribute.

- Syntax

MI_S32 MI_ISP_SetCustSegAttr(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_CustSegAttr_t *pstCustSegAttr);

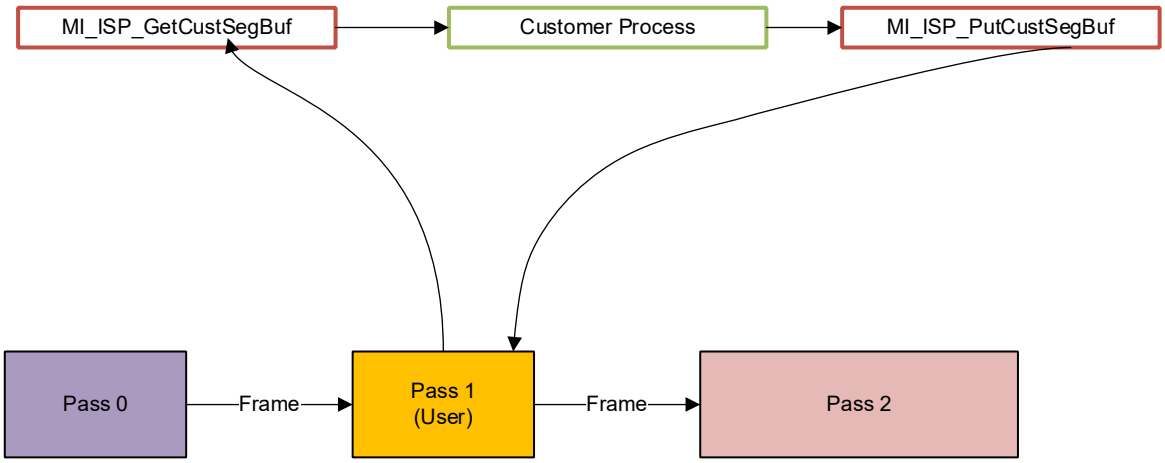
- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstCustSegAttr	AI ISP attribute	Input

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note



When the value of pstCustSegAttr->eMode is E_MI_ISP_CUST_SEG_MODE_INSERT or E_MI_ISP_CUST_SEG_MODE_REPLACE, AI ISP is enabled. MI ISP will be divided into pass0, pass1 and pass2 internally, and the functions are composed of hdr, 3dnr, wdr, rgb2yuv. Pass1 specifically provides raw buffer to the app.

When AI ISP is enabled, isp rotation/mirror/flip is not supported.

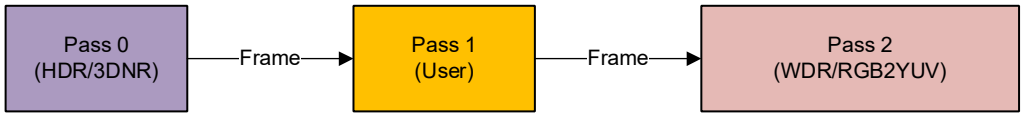
The combination of Pass0 and pass2 is as follows, which can be represented by MI_ISP_CustSegAttr_t.

Note: The pass0 module serial number must be smaller than pass2.

Pass0: HDR,HDR->3DNR,HDR->3DNR->WDR，HDR->3DNR->WDR->RGB2YUV。
Pass2:RGB2YUV,WDR->RGB2YUV,3DNR->WDR->RGB2YUV,HDR->3DNR->WDR->RGB2YUV
E_MI_ISP_SEG_HDR output is before 3DNR.
E_MI_ISP_SEG_3DNR contains DNR Rotation/mirror/flip, and the output is before WDR.
E_MI_ISP_SEG_WDR output is before RGB2YUV.

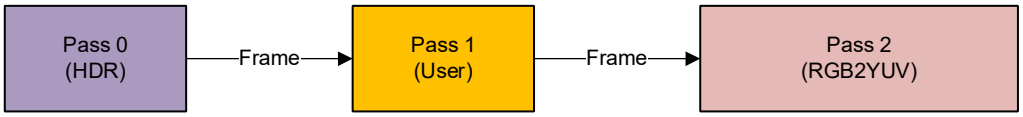
pstCustSegAttr->eMode, pstCustSegAttr->eFrom and pstCustSegAttr->eTo are used to describe the functional composition of pass1.
E_MI_ISP_CUST_SEG_MODE_INSERT: Insert the app image in HDR->3DNR->WDR->RGB2YUV to process pass1.
E_MI_ISP_CUST_SEG_MODE_REPLACE: Replace the function of HDR, 3DNR, WDR, RGB2YUV. You can replace only 3DNR or WDR, or replace both of them.
Example:

a. eFrom= E_MI_ISP_SEG_3DNR,,eTo= E_MI_ISP_SEG_WDR, eMode=E_MI_ISP_CUST_SEG_MODE_INSERT

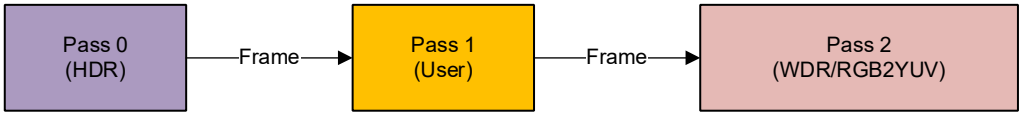


b. eFrom= E_MI_ISP_SEG_3DNR,eTo= E_MI_ISP_SEG_WDR,

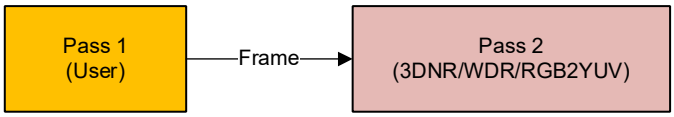
eMode=E_MI_ISP_CUST_SEG_MODE_REPLACE



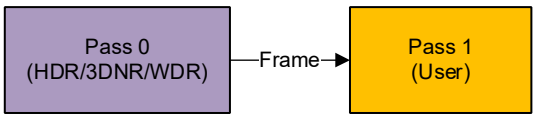
c. eFrom= E_MI_ISP_SEG_3DNR,eTo= E_MI_ISP_SEG_3DNR,
eMode=E_MI_ISP_CUST_SEG_MODE_REPLACE



d. eFrom= E_MI_ISP_SEG_HDR,eTo= E_MI_ISP_SEG_HDR,
eMode=E_MI_ISP_CUST_SEG_MODE_REPLACE



e. eFrom= E_MI_ISP_SEG_RGB2YUV,eTo= E_MI_ISP_SEG_RGB2YUV,
eMode=E_MI_ISP_CUST_SEG_MODE_REPLACE



MI_ISP_SetCustSegAttr should be called after MI_ISP_CreateChannel and before MI_ISP_StartChannel and MI_ISP_SetOutputPortParam.

- Example

MI_ISP initialization:

```
MI_S32 ST_IspModuleInit(MI_ISP_DEV IspDevId)
{
   MI_ISP_CHANNEL IspChnId = 0;
   MI_ISP_PORT  IspOutPortId =1;

     MI_ISP_DevAttr_t stCreateDevAttr;
     memset(&stCreateDevAttr, 0x0, sizeof(MI_ISP_DevAttr_t));
     stCreateDevAttr.u32DevStitchMask = E_MI_ISP_DEVICE_ID0;

      MI_ISP_CreateDevice(IspDevId, &stCreateDevAttr);

      MI_ISP_ChannelAttr_t  stIspChnAttr;
      memset(&stIspChnAttr, 0x0, sizeof(MI_ISP_ChannelAttr_t));
      stIspChnAttr. u32SensorBindId = E_MI_ISP_SENSOR0;
```

```
    MI_ISP_CreateChannel(IspDevId, IspChnId, &stIspChnAttr);

    MI_SYS_WindowRect_t  stInputCropInfo;
    stInputCropInfo.u16x=0;
    stInputCropInfo.u16y=0;
    stInputCropInfo.u16width=0;
    stInputCropInfo.u16height=0;//width/height==0 no use crop.
    MI_ISP_SetInputPortCrop(IspDevId, IspChnId, & stInputCropInfo);

    MI_ISP_ChnParam_t stChnParam;
  stChnParam.eHDRType = E_MI_ISP_HDR_TYPE_OFF;
  stChnParam.e3DNRLevel = E_MI_ISP_3DNR_LEVEL2;
  stChnParam.bMirror = FALSE;
  stChnParam.bFlip = FALSE;
  stChnParam.eRot = E_MI_SYS_ROTATE_NONE;
  stChnParam.bY2bEnable = FALSE;
    MI_ISP_SetChnParam(IspDevId, IspChnId, & stChnParam);

    MI_ISP_CustSegAttr_t stCustSegAttr;
   memset(&stCustSegAttr, 0x0, sizeof(MI_ISP_CustSegAttr_t));
    stCustSegAttr.eMode = E_MI_ISP_CUST_SEG_MODE_INSERT;
    stCustSegAttr.eFrom = E_MI_ISP_SEG_3DNR;
  stCustSegAttr.eTo = E_MI_ISP_SEG_WDR;
  stCustSegAttr.stInputParam.ePixelFormat =
RGB_BAYER_PIXEL(E_MI_SYS_DATA_PRECISION_12BPP, E_MI_SYS_PIXEL_BAYERID_GR);
  stCustSegAttr.stOutputParam.ePixelFormat = stCustSegAttr.stInputParam.ePixelFormat;
    MI_ISP_SetCustSegAttr(enDevId,enChnId, &stCustSegAttr);

    MI_ISP_StartChannel(IspDevId, IspChnId);

    MI_ISP_OutPortParam_t  stIspOutputParam;
    memset(&stIspOutputParam, 0x0, sizeof(MI_ISP_OutPortParam_t));
    stIspOutputParam.stCropRect.u16x=0;
  stIspOutputParam.stCropRect.u16y=0;
  stIspOutputParam.stCropRect.u16Width=0;
  stIspOutputParam.stCropRect.u16Height=0;//width/height use 0, not use crop
    stIspOutputParam.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_YUV420;
    MI_ISP_SetOutputPortParam(IspDevId, IspChnId, IspOutPortId, &stIspOutputParam);
    MI_ISP_EnableOutputPort(IspDevId, IspChnId, IspOutPortId);

  return MI_SUCCESS;
}
```

Get raw:

```
    MI_SYS_BUF_HANDLE hBufHandle = NULL;
    MI_SYS_BufInfo_t stInputBufInfo;
    MI_SYS_BufInfo_t stOutputBufInfo;
    MI_U8 *pSrcData = NULL, *pDstData = NULL;

  memset(&stInputBufInfo, 0, sizeof(MI_SYS_BufInfo_t));
  memset(&stOutputBufInfo, 0, sizeof(MI_SYS_BufInfo_t));
    s32Ret = MI_ISP_GetCustSegBuf(stChnPort.u32DevId, stChnPort.u32ChnId,
&hBufHandle,
                                  &stInputBufInfo, &stOutputBufInfo, 200);
    if(s32Ret == MI_SUCCESS)
    {
     pSrcData = stInputBufInfo.stFrameData->pVirAddr[0];
     pDstData = stOutputBufInfo.stFrameData->pVirAddr[0];
        memcpy(pDstData, pSrcData, stOutputBufInfo.stFrameData.u32BufSize);
        memset(pDstData, 0, stOutputBufInfo.stFrameData.u32BufSize/4);
        MI_ISP_PutCustSegBuf(stChnPort.u32DevId, stChnPort.u32ChnId, hBufHandle);
    }
```

- Related APIs

  None.

2.25. MI_ISP_GetCustSegAttr

- Description

  Get AI ISP attributes.

- Syntax

  MI_S32 MI_ISP_GetCustSegAttr(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
  MI_ISP_CustSegAttr_t *pstCustSegAttr);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pstCustSegAttr	API ISP attribute	output

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h

- Library: libmi_isp.a

※ Note

None.

➢ Example

None.

➢ Related APIs

None.

2.26. MI_ISP_GetCustSegBuf

➢ Description

Get AI ISP buffer.

➢ Syntax

MI_S32 MI_ISP_GetCustSegBuf(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,MI_SYS_BUF_HANDLE *pBufHandle, MI_SYS_BufInfo_t *pstInputBufInfo, MI_SYS_BufInfo_t *pstOutputBufInfo, MI_S32 s32MilliSec);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pBufHandle,	Pass1 input/output Buf handle	Output
pstInputBufInfo	Pass1 input buffer	Output
pstOutputBufInfo	Pass1 output buffer	Output
s32MilliSec	Timeout, unit: ms	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

Used in pairs with MI_ISP_PutCustSegBuf.

➢ Example

Refer to the example of MI_ISP_SetCustSegAttr.

➢ Related APIs

None.

2.27. MI_ISP_PutCustSegBuf

➢ Description

Release AI ISP buffer.

➢ Syntax

MI_S32 MI_ISP_PutCustSegBuf(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_SYS_BUF_HANDLE bufHandle);

➢ Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
ChnId	ISP channel ID	Input
pBufHandle,	Pass1 input/output Buf handle	Input

➢ Return value

- MI_SUCCESS (0) successful.
- Non-Zero: failed, see error code for details.

➢ Dependence

- Header: mi_isp_datatype.h, mi_isp.h
- Library: libmi_isp.a

※ Note

Used in pairs with MI_ISP_GetCustSegBuf.

➢ Example

- Refer to the example of MI_ISP_SetCustSegAttr.
- MI_ISP_PutCustSegBuf should be called in the order as MI_ISP_GetCustSegBuf gets buffer.

➢ Related APIs

None.

2.28. MI_ISP_GetSubChnId

➢ Description

Get ISP subchannel ID

➢ Syntax

MI_S32 MI_ISP_GetSubChnId(MI_ISP_DEV DevId, MI_ISP_CHANNEL MainChnId, MI_ISP_BindSnrId_e eSensorBindId, MI_U32 *pu32SubChnId);

- Parameter

Parameter name	Description	Input/Output
DevId	ISP device ID.	Input
MainChnId	ISP channel ID	Input
eSensorBindId	Sensor ID of subchannel binding	Input
pu32SubChnId	ISP subchannel ID	Output

- Return value
  - MI_SUCCESS (0) successful.
  - Non-Zero: failed, see error code for details.

- Dependence
  - Header: mi_isp_datatype.h, mi_isp.h
  - Library: libmi_isp.a

※ Note
  - In the scene of Stitch, an MI_ISP channel will bind multiple sensors, when IQ wants to control the corresponding sensors, you need to input MI_ISP dev ID, MI_ISP channel ID and sensor ID to get the corresponding subchannel ID, that is, the channel used by IQ.
  - In other cases, the channel used by IQ is the same as MI_ISP channel ID.

- Example

  None.

- Related APIs

  None.

3. ISP DATA TYPE

Video input related data type definition is as following:

Data type	Description
MI_ISP_DEV	Define ISP device type
MI_ISP_CHANNEL	Define ISP channel type
MI_ISP_PORT	Define ISP port type
MI_ISP_DevMaskId_e	Define the device ID used by the mask for the ISP device
MI_ISP_DevAttr_t	Define ISP device attribute parameter
MI_ISP_HDRType_e	Define HDR switch mode
MI_ISP_3DNR_Level_e	Define 3DNR setting level
MI_ISP_BindSnrId_e	Define the sensor ID bound to the ISP
MI_ISP_SYNC3A_e	Synchronize 3A parameters of each channel
MI_ISP_IQApiHeader_t	Define IQ api data type
MI_ISP_VersionPara_t	Define ISP special parameter version
MI_ISP_CustIQParam_t	Define ISP initialization parameter
MI_ISP_ChannelAttr_t	Define ISP channel static attribute parameter
MI_ISP_ChnParam_t	Define ISP channel dynamic attribute parameter
MI_ISP_OutPortParam_t	Define ISP output port parameter
MI_ISP_CALLBK_FUNC	Define callback function type
MI_ISP_CallBackMode_e	Define callback mode
MI_ISP_IrqType_e	Define hardware interrupt type
MI_ISP_CallBackParam_t	Define callback parameter
MI_ISP_ZoomEntry_t	Define the Zoom cell entry type
MI_ISP_ZoomTable_t	Define Zoom Table type
MI_ISP_ZoomAttr_t	Define Zoom attribute
MI_ISP_CustSegAttr_t	Define AI ISP attribute
MI_ISP_CustSegInPortParam_t	Pass1 input port attribute
MI_ISP_CustSegOutPortParam_t	Pass1 output port attribute
MI_ISP_CustSegMode_e	AI ISP mode
MI_ISP_InternalSeg_e	ISP module name

3.1. MI_ISP_DEV

- Description

  Define ISP device type.

- Definition

  typedef MI_U32 MI_ISP_DEV;

※ Note

  None.

- Related data type and interface

  None.

3.2. MI_ISP_CHANNEL

- Description

  Define ISP channel type.

- Definition

  typedef MI_U32 MI_ISP_CHANNEL;

※ Note

  None.

- Related data type and interface

  None.

3.3. MI_ISP_PORT

- Description

  Define ISP port type.

- Definition

  typedef MI_U32 MI_ISP_PORT;

※ Note

  None.

- Related data type and interface

  None.

3.4. MI_ISP_DevMaskId_e

➢ Description

Define the device ID used by the mask for the ISP device.

➢ Definition

```
typedef enum
{
   E_MI_ISP_DEVICEMASK_ID0  = 0x0001,
   E_MI_ISP_DEVICEMASK_ID1  = 0x0002,
   E_MI_ISP_DEVICEMASK_ID_MAX = 0xffff
} MI_ISP_DevMaskId_e;
```

※ Note

ISP E_MI_ISP_DEVICEMASK_ID0 only used by Tiramisu/ Mochi/Maruko.

➢ Related data type and interface

MI_ISP_CreateDevice

3.5. MI_ISP_DevAttr_t

➢ Description

Define ISP device attribute parameter.

➢ Definition

```
typedef struct MI_ISP_DevAttr_s
{
  MI_U32    u32DevStitchMask; //multi ISP  dev bitmask by MI_ISP_DevMaskId_e
}MI_ISP_DevAttr_t;
```

➢ Member

Member name	Description
u32DevStitchMask	ISP DEV ID mask.

※ Note

u32DevStitchMask is composed of MI_ISP_DevMaskId_e and bitmask. If the processing speed of an isp device on the channel cannot meet the requirements, you can use this parameter to reuse another Device, but another device can no longer be created.

➢ Related data type and interface

MI_ISP_CreateDevice

3.6. MI_ISP_HDRType_e

- Description

  Define HDR switch mode.

- Definition

```
typedef enum
{
    E_MI_ISP_HDR_TYPE_OFF,
    E_MI_ISP_HDR_TYPE_VC,
    E_MI_ISP_HDR_TYPE_DOL,
    E_MI_ISP_HDR_TYPE_EMBEDDED,
    E_MI_ISP_HDR_TYPE_LI,
    E_MI_ISP_HDR_TYPE_MAX
} MI_ISP_HDRType_e;
```

- Member

Member name	Description
E_MI_ISP_HDR_TYPE_OFF	Turn off HDR
E_MI_ISP_HDR_TYPE_VC	virtual channel mode HDR, vc0->long,vc1->short
E_MI_ISP_HDR_TYPE_DOL	Digital Overlap High Dynamic Range
E_MI_ISP_HDR_TYPE_EMBEDDED	compressed HDR mode
E_MI_ISP_HDR_TYPE_LI	Line interlace HDR
E_MI_ISP_HDR_TYPE_MAX	HDR boundary value

※ Note

The HDR Type used can be obtained through the MI_SNR_GetPadInfo interface.
The HDR function requires the cooperation of MI_VIF and MI_ISP modules, and the same HDR type needs to be set on both sides.

- Related data type and interface

  MI_ISP_ChnParam_t

3.7. MI_ISP_3DNR_Level_e

- Description

  Define 3DNR setting level.

- Definition

```
typedef enum
{
    E_MI_ISP_3DNR_LEVEL_OFF,
    E_MI_ISP_3DNR_LEVEL1,
```

```
    E_MI_ISP_3DNR_LEVEL2,
    E_MI_ISP_3DNR_LEVEL3,
    E_MI_ISP_3DNR_LEVEL4,
    E_MI_ISP_3DNR_LEVEL5,
    E_MI_ISP_3DNR_LEVEL6,
    E_MI_ISP_3DNR_LEVEL7,
    E_MI_ISP_3DNR_LEVEL_NUM
}MI_ISP_3DNR_Level_e;
```

※ Note

The higher the level, the better the noise reduction effect and the more buffer consumed.

Chip	3DNR support max level
Tiramisu	E_MI_ISP_3DNR_LEVEL2
Muffin	E_MI_ISP_3DNR_LEVEL2
Mochi	E_MI_ISP_3DNR_LEVEL7
Maruko	E_MI_ISP_3DNR_LEVEL7

➢ Related data type and interface

MI_ISP_ChnParam_t

3.8. MI_ISP_BindSnrId_e

➢ Description

Define the sensor ID bound to the ISP.

➢ Definition

```
typedef enum
{
    E_MI_ISP_SENSOR_INVALID = 0,
    E_MI_ISP_SENSOR0 = 0x1,
    E_MI_ISP_SENSOR1 = 0x2,
    E_MI_ISP_SENSOR2 = 0x4,
    E_MI_ISP_SENSOR3 = 0x8,
    E_MI_ISP_SENSOR4 = 0x10,
    E_MI_ISP_SENSOR5 = 0x20,
    E_MI_ISP_SENSOR6 = 0x40,
    E_MI_ISP_SENSOR7 = 0x80,
    E_MI_ISP_SENSOR_MAX = 8
}MI_ISP_BindSnrId_e;
```

※ Note

None.

➢ Related data type and interface

None.

3.9. MI_ISP_SYNC3A_e

- Description

  Synchronize 3A parameters of each channel.

- Definition

```
typedef enum
{
  E_MI_ISP_SYNC3A_NONE = 0x00,
  E_MI_ISP_SYNC3A_AE   = 0x01,
  E_MI_ISP_SYNC3A_AWB  = 0x02,
  E_MI_ISP_SYNC3A_IQ   = 0x04
}MI_ISP_SYNC3A_e;
```

- Member

Member name	Description
E_MI_ISP_SYNC3A_NONE	Async 3A
E_MI_ISP_SYNC3A_AE	Sync AE
E_MI_ISP_SYNC3A_AWB	Sync AWB
E_MI_ISP_SYNC3A_IQ	Sync IQ parameter setting

※ Note

None.

- Related data type and interface

None.

3.10. MI_ISP_IQApiHeader_t

- Description

Define IQ api data type.

- Definition

```
typedef struct MI_ISP_IQApiHeader_s
{
   MI_U32 u32HeadSize;    //Size of MIIspApiHeader_t
   MI_U32 u32DataLen;     //Data length;
   MI_U32 u32CtrlID;      //Function ID
   MI_U32 u32Channel;     //Isp channel number
   MI_U32 u32DevId;       //Isp Device Id
   MI_S32 s32Ret;         //Isp api retuen value
} MI_ISP_IQApiHeader_t;
```

- Member

Member name	Description
u32HeadSize	MI_ISP_IQApiHeader_t struct takes up space
u32DataLen	Data size
u32CtrlID	Function corresponding control Id
u32Channel	Isp channel Id
u32DevId	Isp device Id
s32Ret	Interface return value

※ Note

None.

➢ Related data type and interface

None.

3.11. MI_ISP_VersionPara_t

➢ Description

Define ISP special parameter version.

➢ Definition

```
typedef struct MI_ISP_VersionPara_s
{
    MI_U32 u32Revision;
    MI_U32 u32Size;
    MI_U8 u8Data[VERSIONPARA_DATA_SIZE];
}MI_ISP_VersionPara_t;
```

➢ Member

Member name	Description
u32Revision	Negotiate with IQ team to define the corresponding version number
u32Size	Data corresponds to effective size
u8Data[VERSIONPARA_DATA_SIZE]	Store data, the value of VERSIONPARA_DATA_SIZE is 64

※ Note

None.

➢ Related data type and interface

MI_ISP_CustIQParam_t

3.12. MI_ISP_CustIQParam_t

➢ Description

Define ISP initialization parameter.

➢ Definition

```
typedef struct MI_ISP_CustIQParam_s
{
   MI_ISP_VersionPara_t  stVersion;
}MI_ISP_CustIQParam_t;
```

➢ Member

Member name	Description
stVersion	Version parameter

※ Note

Some IQ initialization parameters that require ISP customized processing can be set through this parameter. If there is no requirement, you can set all of them to 0.

➢ Related data type and interface

None.

3.13. MI_ISP_ChannelAttr_t

➢ Description

Define ISP channel static attribute parameter.

➢ Definition

```
typedef struct MI_ISP_ChannelAttr_s
{
   MI_U32  u32SensorBindId; //bitmask by MI_ISP_BindSnrId_e
   MI_ISP_CustIQParam_t stIspCustIqParam;
   MI_U32  u32Sync3AType; //sync 3a bitmask by MI_ISP_SYNC3A_e
}MI_ISP_ChannelAttr_t;
```

➢ Member

Member name	Description
u32SensorBindId	Sensor Id corresponding to channel binding
stIspCustIqParam	Isp IQ initialization parameter
u32Sync3AType	Sync the 3A/IQ parameters between multiple groups of images in the channel

※ Note

- When a multi-sensor splicing scene, one ISP channel needs to process multiple sensor images, you need to set up multiple sensor Id bitmasks through u32SensorBindId, otherwise only the corresponding sensor Id interface is set. If you do not need ISP to do IQ/3A, or the front-end data source is not sensor, you can set it to 0.
- When multi-sensor splicing scenes, one ISP channel needs to process multiple sensor images, set the 3A/IQ effect synchronization between multiple images through u32Sync3AType to avoid

excessive differences in effects between the two images.

- Related data type and interface

  MI_ISP_CreateChannel

3.14. MI_ISP_ChnParam_t

- Description

  Define ISP channel dynamic attribute parameter.

- Definition

```
typedef struct MI_ISP_ChnParam_s
{
  MI_ISP_HDRType_e      eHDRType;
  MI_ISP_3DNR_Level_e   e3DNRLevel;
  MI_BOOL               bMirror;
  MI_BOOL               bFlip;
  MI_SYS_Rotate_e       eRot;
   MI_BOOL              bY2bEnable;
} MI_ISP_ChnParam_t;
```

- Member

Member name	Description
eHDRType	HDR switch parameter
e3DNRLevel	3dnr lever
bMirror	Enable channel mirror
bFlip	Enable channel flip
eRot	Channel rotation parameter
bY2bEnable	Enable the conversion function of yuv to bayer

※ Note

- Non Maruko chip, when eRot > 0 or bflip = true, e3dnrlevel must be greater than 0.
- bY2bEnable is only supported by Muffin and only supported when the input port is Frame Mode.
- Mochi do not support HDR/Mirror/Flip/Rot/Y2bEnable.

- Related data type and interface

  MI_ISP_SetChnParam

3.15. MI_ISP_OutPortParam_t

- Description

  Define ISP output port parameter.

- Definition

```
typedef struct MI_ISP_OutPortParam_s
{
    MI_SYS_WindowRect_t stCropRect;
      MI_SYS_PixelFormat_e  ePixelFormat;
      MI_SYS_CompressMode_e eCompressMode;
}MI_ISP_OutPortParam_t;
```

➢ Member

Member name	Description
stCropRect	Output crop area
ePixelFormat	Pixel format
eCompressMode	Output compress mode

※ Note

- eCompressMode support range

Chip	Output support compress mode
Tiramisu	E_MI_SYS_COMPRESS_MODE_NONE
Muffin	E_MI_SYS_COMPRESS_MODE_NONE
Mochi	E_MI_SYS_COMPRESS_MODE_NONE E_MI_SYS_COMPRESS_MODE_TO_6BIT
Maruko	E_MI_SYS_COMPRESS_MODE_TO_6BIT

➢ Related data type and interface

None.

3.16. MI_ISP_CALLBK_FUNC

➢ Description

Define callback function type.

➢ Definition

```
typedef MI_S32 (*MI_ISP_CALLBK_FUNC)(MI_U64 u64Data);
```

※ Note

None.

➢ Related data type and interface

MI_ISP_CallBackParam_t

3.17. MI_ISP_CallBackMode_e

➢ Description

Define callback mode.

➢ Definition

```
typedef enum
{
    E_MI_ISP_CALLBACK_ISR,
    E_MI_ISP_CALLBACK_MAX,
} MI_ISP_CallBackMode_e;
```

➢ Member

Member name	Description
E_MI_ISP_CALLBACK_ISR	Hardware interrupt mode callback
E_MI_ISP_CALLBACK_MAX	Maximum value of callback mode

※ Note

Currently only supports ISR callback mode.

➢ Related data type and interface

MI_ISP_CallBackParam_t

3.18. MI_ISP_IrqType_e

➢ Description

Define hardware interrupt type.

➢ Definition

```
typedef enum
{
    E_MI_ISP_IRQ_ISPVSYNC,
    E_MI_ISP_IRQ_ISPFRAMEDONE,
    E_MI_ISP_IRQ_MAX,
} MI_ISP_IrqType_e;
```

➢ Member

Member name	Description
E_MI_ISP_IRQ_ISPVSYNC	ISP Vsync interrupt type
E_MI_ISP_IRQ_ISPFRAMEDONE	ISP Frame done interrupt type
E_MI_ISP_IRQ_MAX	ISP hardware interrupt type maximum

※ Note

- E_MI_VPE_IRQ_ISPVSYNC: The signal of the first pixel in each frame.
- E_MI_VPE_IRQ_ISPFRAMEDONE：The signal at the end of each frame written by isp.

➢ Related data type and interface

MI_ISP_CallBackParam_t

3.19. MI_ISP_CallBackParam_t

- Description

  Define callback parameter.

- Definition

```
typedef struct MI_ISP_CallBackParam_s
{
    MI_ISP_CallBackMode_e eCallBackMode;
    MI_ISP_IrqType_e  eIrqType;
    MI_ISP_CALLBK_FUNC  pfnCallBackFunc;
    MI_U64    u64Data;
} MI_ISP_CallBackParam_t;
```

- Member

Member name	Description
eCallBackMode	Callback mode
eIrqType	Hardware interrupt mode type
pfnCallBackFunc	Callback interface pointer
u64Data	Callback interface parameter

※ Note

None.

- Related data type and interface

MI_ISP_CallBackTask_Register
MI_ISP_CallBackTask_Unregister

3.20. MI_ISP_ZoomEntry_t

- Description

  Define the Zoom cell entry type.

- Definition

```
typedef struct MI_ISP_ZoomEntry_s
{
    MI_SYS_WindowRect_t stCropWin;
    MI_U8 u8ZoomSensorId;
} MI_ISP_ZoomEntry_t;
```

- Member

Member name	Description
stCropWin	Crop position parameter
u8ZoomSensorId	The sensor Id corresponding to the Crop parameter

※ Note

u8ZoomSensorId corresponds to the return value of the pCus_sensor_GetCurSwtichSensorId callback function in the sensor driver.

➢ Related data type and interface

MI_ISP_ZoomTable_t

3.21. MI_ISP_ZoomTable_t

➢ Description

Define Zoom Table type.

➢ Definition

```
typedef struct MI_ISP_ZoomTable_s
{
    MI_U32 u32EntryNum;
    MI_ISP_ZoomEntry_t *pVirTableAddr;
} MI_ISP_ZoomTable_t;
```

➢ Member

Member name	Description
u32EntryNum	Number of entries in the Zoom Table
pVirTableAddr	Zoom Table Buffer Pointer

※ Note

None.

➢ Related data type and interface

MI_ISP_LoadPortZoomTable

3.22. MI_ISP_ZoomAttr_t

➢ Description

Define Zoom attribute.

➢ Definition

```
typedef struct MI_ISP_ZoomAttr_s
{
    MI_U32  u32FromEntryIndex;
    MI_U32  u32ToEntryIndex;
    MI_U32  u32CurEntryIndex;
} MI_ISP_ZoomAttr_t;
```

➢ Member

Member name	Description

u32FromEntryIndex	Start to run Zoom entry index
u32ToEntryIndex	Stop to run Zoom entry index
u32CurEntryIndex	Index of current Zoom position

※ Note

None.

➢ Related data type and interface

MI_ISP_StartPortZoom

MI_ISP_GetPortCurZoomAttr

3.23. MI_ISP_CustSegAttr_t

➢ Description

Define AI ISP attribute.

➢ Definition

```
typedef struct MI_ISP_CustSegAttr_s
{
    MI_ISP_CustSegMode_e        eMode;
    MI_ISP_InternalSeg_e        eFrom;
    MI_ISP_InternalSeg_e        eTo;
    MI_ISP_CustSegInPortParam_t  stInputParam;
    MI_ISP_CustSegOutPortParam_t stOutputParam;
} MI_ISP_CustSegAttr_t;
```

➢ Member

Member name	Description
eMode	Pass1 mode
eFrom	Head module of Pass1
eTo	End module of Pass1
stInputParam	Pass1 input port attribute
stOutputParam	Pass1 output port attribute

※ Note

For parameter function, please refer to the description of MI_ISP_SetCustSegAttr.

➢ Related data type and interface

MI_ISP_SetCustSegAttr

MI_ISP_GetCustSegAttr

3.24. MI_ISP_CustSegInPortParam_t

➢ Description

Pass1 input port attribute.

➢ Definition

```
typedef struct MI_ISP_CustSegInPortParam_s
{
 MI_SYS_PixelFormat_e ePixelFormat;
} MI_ISP_CustSegInPortParam_t;
```

➢ Member

Member name	Description
ePixelFormat	Data format

※ Note

Before 3DNR only supports 12bit bayer and 16bit bayer input and output.
Before WDR only supports 16bit bayer input and output.
Before RGB2YUV only supports the input and output of E_MI_SYS_PIXEL_FRAME_ARGB8888 and E_MI_SYS_PIXEL_FRAME_RGB101010.

➢ Related data type and interface

MI_ISP_SetCustSegAttr
MI_ISP_GetCustSegAttr

3.25. MI_ISP_CustSegOutPortParam_t

➢ Description

Pass1 output port attribute.

➢ Definition

```
typedef struct MI_ISP_CustSegOutPortParam_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
} MI_ISP_CustSegOutPortParam_t;
```

➢ Member

Member name	Description
ePixelFormat	Data format

※ Note

- Before 3DNR only supports 12bit bayer and 16bit bayer input and output.
- Before WDR only supports 16bit bayer input and output.
- Before RGB2YUV only supports the input and output of E_MI_SYS_PIXEL_FRAME_ARGB8888 and E_MI_SYS_PIXEL_FRAME_RGB101010.

➢ Related data type and interface

MI_ISP_SetCustSegAttr
MI_ISP_GetCustSegAttr

3.26. MI_ISP_CustSegMode_e

- Description

  AI ISP mode.

- Definition

```
typedef enum
{
    E_MI_ISP_CUST_SEG_MODE_NONE = 0,
    E_MI_ISP_CUST_SEG_MODE_INSERT,
    E_MI_ISP_CUST_SEG_MODE_REPLACE,
    E_MI_ISP_CUST_SEG_MODE_MAX
} MI_ISP_CustSegMode_e;
```

- Member

Member name	Description
E_MI_ISP_CUST_SEG_MODE_NONE	Disable AI ISP
E_MI_ISP_CUST_SEG_MODE_INSERT	Insert mode
E_MI_ISP_CUST_SEG_MODE_REPLACE	Replacement mode
E_MI_ISP_CUST_SEG_MODE_MAX	Maximum value

※ Note

None.

- Related data type and interface

MI_ISP_SetCustSegAttr
MI_ISP_GetCustSegAttr

3.27. MI_ISP_InternalSeg_e

- Description

  ISP module name.

- Definition

```
typedef enum
{
    E_MI_ISP_SEG_INVALID = 0,
    E_MI_ISP_SEG_HDR,     // HDR
```

```
    E_MI_ISP_SEG_3DNR,    // 3DNR
    E_MI_ISP_SEG_WDR,     // WDR
    E_MI_ISP_SEG_RGB2YUV, // RGB2YUV
    E_MI_ISP_SEG_MAX
} MI_ISP_InternalSeg_e;
```

- Member

Member name	Description
E_MI_ISP_SEG_INVALID	Invalid module
E_MI_ISP_SEG_HDR	HDR module
E_MI_ISP_SEG_3DNR	DNR,Rotation/mirror/flip module
E_MI_ISP_SEG_WDR	WDR module
E_MI_ISP_SEG_RGB2YUV	RGB2YUV module
E_MI_ISP_SEG_MAX	Maximum value

※ Note

None.

- Related data type and interface

MI_ISP_SetCustSegAttr

MI_ISP_GetCustSegAttr

4. ISP ERROR CODE

API error codes are as the table shown below:

Error code	Macro definition	Description
0xA0078001	MI_ERR_ISP_INVALID_CHNID	Invalid device channel ID
0xA0078002	MI_ERR_ISP_INVALID_PORTID	Invalid device port ID
0xA0078003	MI_ERR_ISP_ILLEGAL_PARAM	Illegal parameter
0xA0078004	MI_ERR_ISP_EXIST	Device has exited
0xA0078005	MI_ERR_ISP_UNEXIST	Device has not exited
0xA0078006	MI_ERR_ISP_NULL_PTR	Null pointer parameter
0xA0078008	MI_ERR_ISP_NOT_SUPPORT	Not supported
0xA0078009	MI_ERR_ISP_NOT_PERM	Operation is not permitted
0xA007800C	MI_ERR_ISP_NOMEM	Failed to allocate memory
0xA007800D	MI_ERR_ISP_NOBUF	Memory has run out
0xA007800E	MI_ERR_ISP_BUF_EMPTY	Video input buffer is empty
0xA0078010	MI_ERR_ISP_NOTREADY	Device is not initialized
0xA0078012	MI_ERR_ISP_BUSY	Device system is busy

5. PROCFS INTRODUCTION

5.1. cat

- Debug info

  # cat /proc/mi_modules/mi_isp/mi_isp0

```
------------------------- end dump ISP Dev info --------------------------------
 DevID  CreChnNum  Devmask       cmdq  En  Mode  num  VsyncCnt  FrameDoneCnt  DropCnt
     0         2        1   c662b788   1     1   27        370           371        0
------------------------- End dump ISP dev 0 info -------------------------------

------------------------- start dump ISP CHN info -------------------------------
 DevId  ChnId  start  SnrId  sync3A  chnNum                Crop     InSize     pixel  Stride  Atom  Atom0
     0      0      1      0       0       1(   0,   0,1280, 720)(1280, 720)YUV422UYVY       0     0    185
     0      1      1      0       0       1(   0,   0,1280, 720)(1280, 720)YUV422UYVY       0     0    185

 DevId  ChnId  Rot  bMirr/flip  3DNRLevel  HdrMode
     0      0   0(   0,    0)          2        0
     0      1   0(   0,    0)          2        0

  DevId  ChnId  PreCnt  EnqCnt  BarCnt  checkin  checkout  DeqCnt  DropCnt  EnqOTNull
     0      0     185     185     185      185       185     185        5          0
     0      1     185     185     185      185       185     185        4          0
------------------------- End dump ISP CHN info --------------------------------

------------------------- start dump ISP OUTPUT PORT info ----------------------
 DevId  ChnId  PortId  bindtype  Enable     Pixel            PortCrop  Stride  GetCnt  FailCnt  FinishCnt  fps
     0      0       0      Real       1YUV422UYVY(   0,   0,1280, 720)       0     185        0        185    25.00
     0      0       1     Frame       0YUV422YUYV(   0,   0,   0,   0)       0       0        0          0     0.00
     0      0       2     Frame       0YUV422YUYV(   0,   0,   0,   0)       0       0        0          0     0.00
     0      1       0      Real       1YUV422UYVY(   0,   0,1280, 720)       0     185        0        185    25.00
     0      1       1     Frame       0YUV422YUYV(   0,   0,   0,   0)       0       0        0          0     0.00
     0      1       2     Frame       0YUV422YUYV(   0,   0,   0,   0)       0       0        0          0     0.00
------------------------- End dump ISP OUTPUT PORT info ------------------------
```

- Debug info analysis

  Record the current ISP usage status and related attributes, you can get this information dynamically, which is convenient for debugging and testing.

- Parameter Description

Parameter		**Description**
ISP Dev info	DevId	Isp Dev Id
	CreChnNum	Chn number
	DevMask	Stitch Dev Mask
	cmdq	Cmdq pointer
	En	ISR interrupt enable flag
	Mode	ISR interrupt mode
	num	ISR ID

Parameter		Description
	VsyncCnt	ISR ID
	FrameDoneCnt	ISR Frame Done Cnt
	DropCnt	ISR Frame Drop Cnt

Parameter		Description
ISP Chn info	DevId	Isp Dev Id
	ChnId	Isp Chn Id
	start	Chn Start Flag
	SnrId	Bind sensorId
	Sync3A	Sync 3A type
	chnNum	Muti chn Num
	Crop	Input Crop Size
	InSize	Input Size
	pixel	Input pixel format
	Stride	Input Buffer Stride
	Atom	Bottom buffers
	Atom0	There is no buffer at the bottom after the buffer is released
	Rot	Rotation angle
	bMirror/Flip	mirror/flip
	3DNRLevel	3DNR Level
	HdrMode	HDR mode
	PreCnt/EnqCnt/ BarCnt/ checkin/checkout/ DeqC nt/DropCnt	Callback execution times
	EnqOTNull	Statistics of times that OutBuffer is Null in Enq

Parameter		Description
ISP OutPut Port info	DevId	Dev Id
	ChnId	Plane id
	PortID	Port Id
	Bindtype	Binding mode with post-level
	Enable	Port enable flag
	Pixel	Port output pixel format

Parameter		Description
	PortCrop	output Crop Size
	Stride	Output Stride
	GetCnt	Count of getting Output buffers
	Failcnt	Count of failed to get Output buffers
	FinishCnt	Count of finished output buffers
	fps	Output port frame rate

5.2. echo

echo help > /proc/mi_modules/mi_isp/mi_isp0

```
debugmutex      [/];

stopchnl        [chnid ON/OFF];

dumptaskfile    [chnid Cnt /path/ passid bOnlyDumpResChange bdumpport];

setatom         [chnid AtomValue];

clearbuf        [chnid portid bClearInput bclearoutput bcleardonebuff passid];

debuglv         [level](1:isp api,2:flow,4:check irq done)
                    [chnid][level] (1:check buffer,2:check loop,4:check frame pts,8:check fence done,16:check func time,32:check sidebandmsg)

enableport      [chnid portid bEn];

skipframe       [chnid skipnum];

protectport     [chnid portid clientid bEn passid];
                    clientid:isp:0x23, LDC:0x0e, scl(0,1,2):0x19, 0x1c, 0x1d

clearbindq      [chnid bclear passid];

fpsth           [chnid portid fpsint fpsfloat passid];

outputparam     [pixel chnid portid pixelmode];

outputparam     [compress chnid portid compressmode];

outputparam     [crop chnid portid cropX cropY cropW cropH];

outputparam     [all chnid portid pixelmode compressmode cropX cropY cropW cropH];
```

Echo help can view the available commands.

Function	Print all functions/line/time used by the channel locks.
Command	echo debugmutex > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	None
Example	None

Function	Stop the specified channel.
Command	echo stopchnl [ChnID] [ON/OFF] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	[ChnId] : Channel ID [ON/OFF] ON:stop chn OFF:start chn

Example	echo stopchnl 0 ON > /proc/mi_modules/mi_isp/mi_isp0

Function	Dump the output data of the selected channel and save it in /path.
Command	echo dumptaskfile [chnid, Cnt, /path/, bOnlyDumpResChange, bdumpport] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: Channel id
	Cnt: Dump count
	Path: Path to store dump files
	bOnlyDumpResChange: Blurred images may be generated when switching resolutions. The dump command and switching resolution operations are not easy to perform together. You can set this parameter first, and then switch the resolution. After the program recognizes it, the buffer after the resolution change will be dumped. (Optional)
	Bdumpport: Only dump a certain portid, the default is to dump all the ports in the output. (Optional)
Example	echo dumptaskfile 0 1 /mnt > /proc/mi_modules/mi_isp/mi_isp0

Function	Set Dev atomvalue.
Command	echo setatom [AtomValue] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	AtomValue: The maximum number of buffers held by the Driver in input realtime mode
Example	echo setatom 3 > /proc/mi_modules/mi_isp/mi_isp0 ps: Increase the atom when the bottom layer is stuck, add a buffer to the driver, and re-trig to see if it recovers, or increase it when the frame is dropped to see if the frame can be full.

Function	Clear the input/output/done of the specified channel portbuffer.
Command	echo clearbuf [chnid, portid bClearInput, bclearoutput, bcleardonebuff] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	Portid: port id
	bClearInput: Clear input buffer
	bclearoutput: Clear output buffer
	cleardonebuff: Clear done buffer
Example	echo clearbuf 0 0 1 0 0 > /proc/mi_modules/mi_isp/mi_isp0

Function	Set debug level.
Command	echo debuglv [level] >/proc/mi_modules/mi_isp/mi_isp0

	echo debuglv [chnid][level] >/proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	Level: Single level parameter: 1:isp api,2:flow,4:check irq done Double parameter: 1:check buffer,2:check loop,4:check frame pts,8:check fence done,16:check func time,32:check sidebandmsg,64:check zoominfo
Example	echo debuglv 4 > /proc/mi_modules/mi_isp/mi_isp0 // Open check irq done related MI print
	echo debuglv 0 3 > /proc/mi_modules/mi_isp/mi_isp0 // Open channel0 level3 printing to print buffer info and loop info

Function	Drop frame count
Command	echo skipframe [chnid, skipnum] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	skipnum: drop frame count
Example	echo skipframe 0 60 > /proc/mi_modules/mi_isp/mi_isp0

Function	Protect outputport buffer
Command	echo protectport [chnid portid clientid ben] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	Portid: port id
	Clientid: isp:0x23, LDC:0x0e, scl(0,1,2):0x19, 0x1c, 0x1d
	ben: Enable
Example	echo protectport 0 0 0x23 1 > /proc/mi_modules/mi_isp/mi_isp0 // Protect isp in channel0 pass0 port0

Function	Clear Bind Q buffer
Command	echo clearbindq [chnid bclear] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	bclear: Whether to clear
Example	echo clearbindq 0 1 > /proc/mi_modules/mi_isp/mi_isp0 //Clear channel0 bindQ buffer The ISP BindQ is full and the front-end cannot get the buffer, and judge whether the ISP is too slow. Clear bindQ and observe whether it will accumulate. If it is, the ISP is too slow, if not, the previous exception is not consumed.

Function	Set the print frame rate, print if it is lower than this value
Command	echo fpsth [chnid portid fpsint fpsfloat] > /proc/mi_modules/mi_isp/mi_isp0

Parameter Description	Chnid: channel id
	Portid: port id
	fpsint: Frame rate integer bits
	fpsfloat: Frame rate demical bits
Example	echo fpsth 0 0 30 30 > /proc/mi_modules/mi_isp/mi_isp0 // Print the fps which is less than 30.30 in channel0 port0

Function	Enable port or not
Command	echo enableport [chnid, portid, bEn] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	Portid: port id
	bEn: 1/0
Example	echo enableport 0 0 1 > /proc/mi_modules/mi_isp/mi_isp0

Function	set output port param
Command	echo outputparam pixel [chnid, portid pixelmode] > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam compress [chnid, portid compressmode] > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam crop [chnid, portid cropX cropY cropW cropH] > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam all [chnid, portid pixelmode compressmode cropX cropY cropW cropH] > /proc/mi_modules/mi_isp/mi_isp0
Parameter Description	Chnid: channel id
	Portid: port id
	Pixelmode: output pixel format
	Compressmode：output compress mode
	cropX：output crop X
	cropY：output crop Y
	cropW：output crop width
	cropH：output crop height
Example	echo outputparam pixel 0 1 11 > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam compress 0 1 5 > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam crop 0 1 0 0 1920 1080 > /proc/mi_modules/mi_isp/mi_isp0
	echo outputparam all 0 1 11 5 0 0 1920 1080 > /proc/mi_modules/mi_isp/mi_isp0